

Sleeping Machines: what is known

Computing in time with races, holds and vetoes · report, 27 September 2026

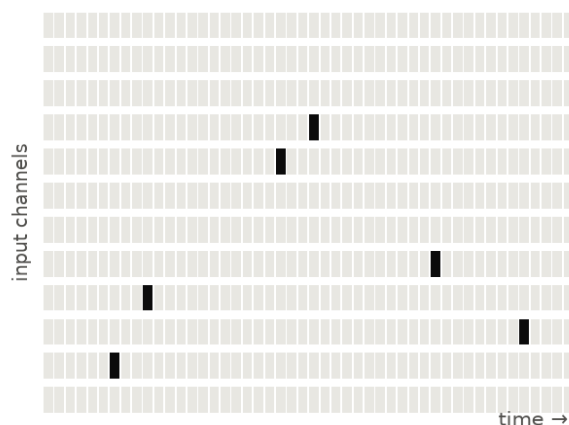
In plain terms

Today's neural networks are **clocked and dense**: at every step, every input is multiplied by every weight, whether or not anything happened. Many real signals are the opposite: long silences broken by precisely timed events (nerve spikes, trades on a market, sensor alarms), where *when* something happens is the information.

Sleeping Machines are networks that only work when an event arrives. A node waits. It fires when the right inputs arrive in the right time window ("B within 1.5 s after A"), and the first node to fire gives the answer, a *race*. Silence costs nothing, and time itself does the computing: a delay or a waiting window plays the role that a weight matrix plays in a dense network.

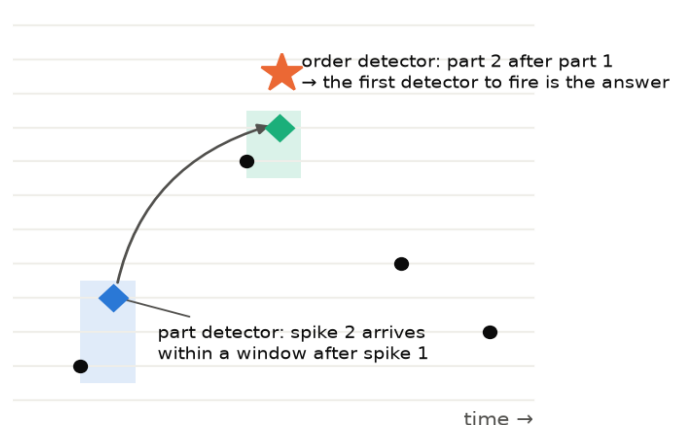
The questions are whether such networks can **learn** (without backpropagation or any dense machinery: a node adjusts only its few connections that were active, like moving money between accounts under a fixed budget) and whether they can **match or beat** MLPs and Transformers.

Clocked network: every cell, every tick



576 cells × every weight $\approx 10^5$ – 10^6 multiply-adds

Event network: work only when a spike arrives

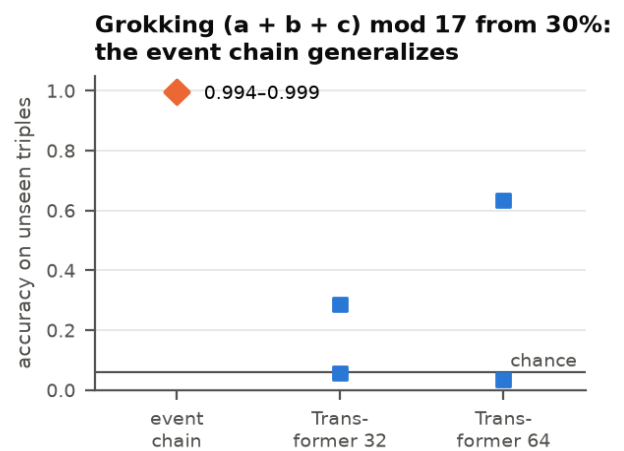
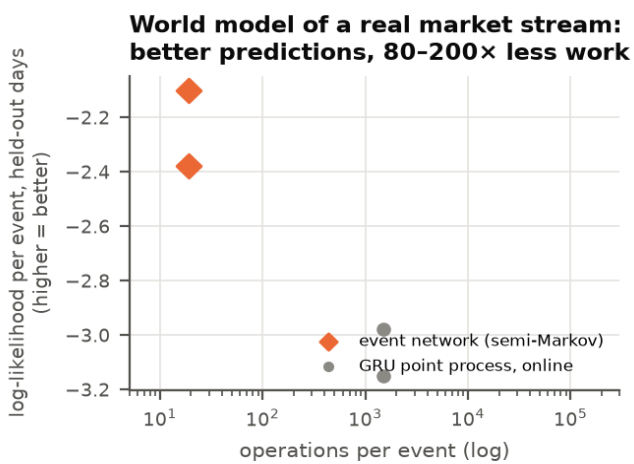
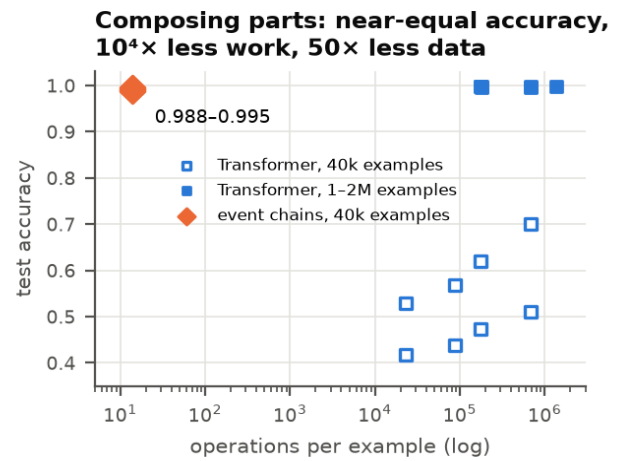
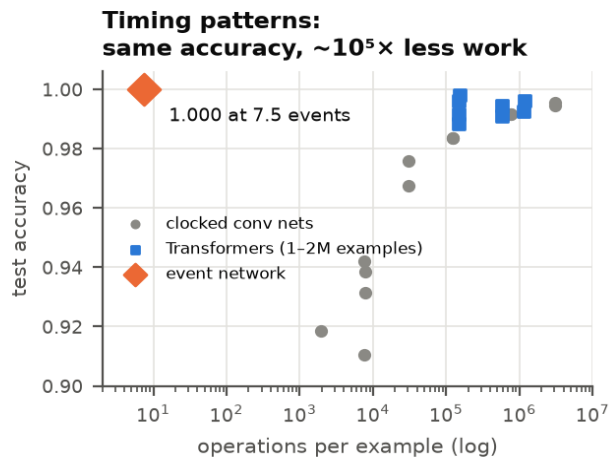


≈ 10 events; silence costs nothing

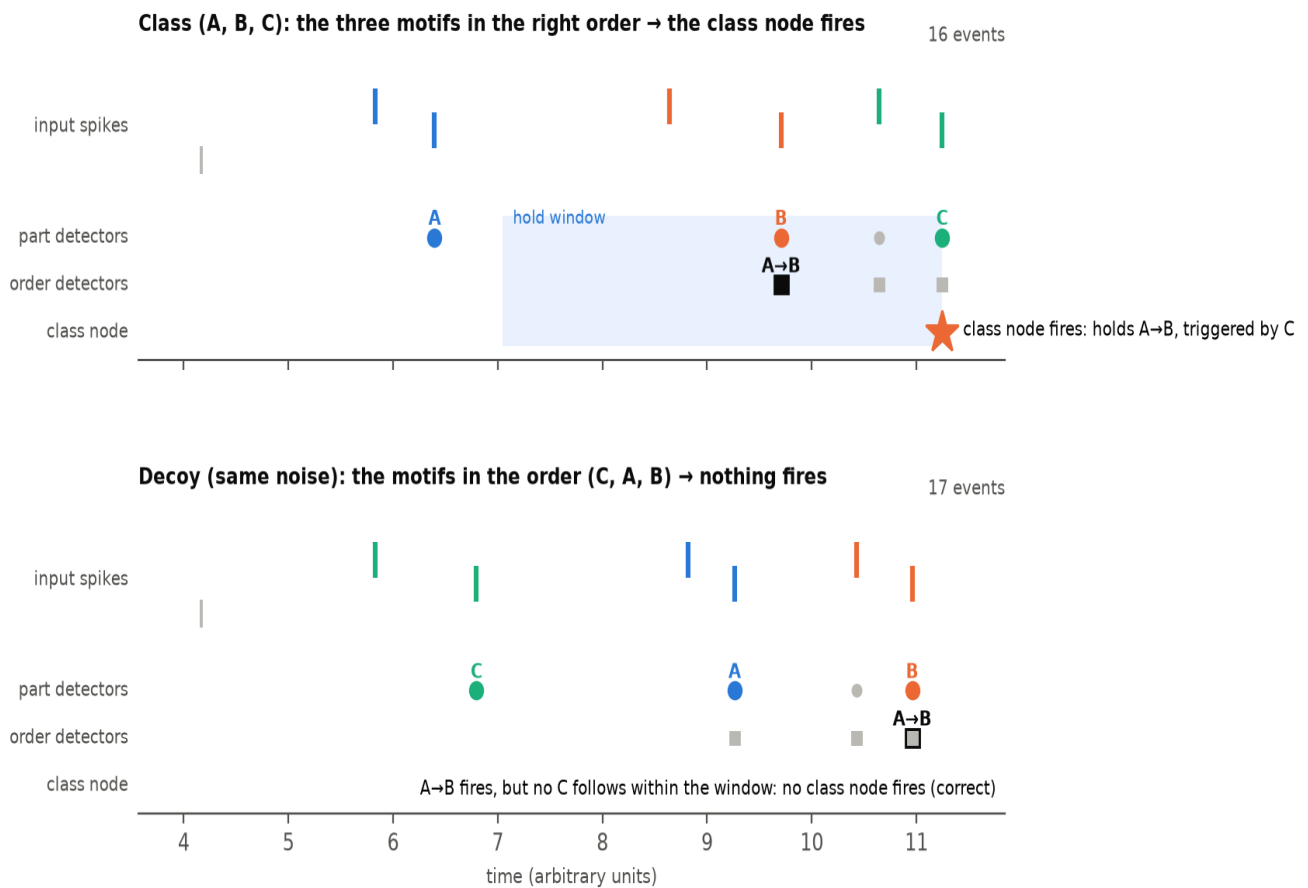
Highlights

- **Same accuracy, 10,000–100,000× less computation.** On timing-pattern recognition a learned event network is perfect (1.000) using ≈ 7.5 events per example; Transformers reach 0.989–0.998 at 150k–1.2M multiply-adds after 1–2M training examples.
- **It groks where a Transformer does not.** Trained on 30% of all (a, b, c) triples, it learns (a + b + c) mod 17 and is 99.4–99.9% correct on the triples it never saw; a Transformer with weight decay stays at 3–63%.
- **Deep order from a few thousand examples.** Recognizing which of 20 orders of four patterns occurred needs four levels of “this, then that”. The network finds the right detectors among 55 million candidates and is 99.9–100% correct on 5 of 5 runs after 10–15k examples, with $\approx 2,000$ learning updates, ≈ 150 events per example and only $\approx 80k$ connections ever created (Transformer comparison on this task running).
- **A better world model of a real market stream at 80–200× less computation.** Predicting the next trade events of BTC on days it never saw, a small event network beats a recurrent neural point process (GRU).
- **Learning cost follows activity, not size.** Eight times more inputs (12 → 96 channels) costs no more learning mistakes.
- **New theory, proved:** exactly what one event node can compute and where depth is needed; why a fixed weight budget lets a node learn an AND without knowing which half was wrong; why learning

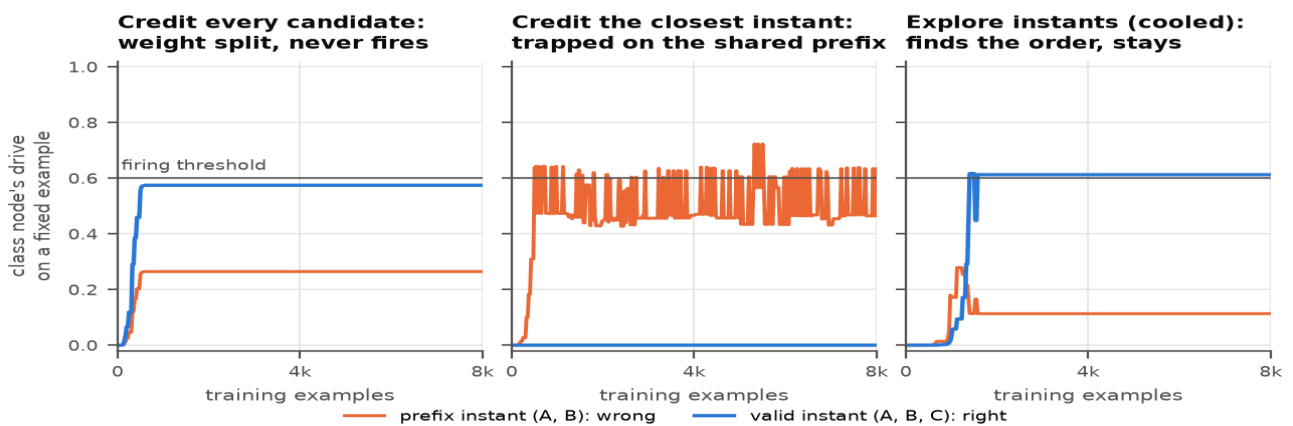
deep order needs a little exploration and a safety margin.



What the network actually does on one example: spikes arrive; part detectors fire when two spikes are close enough in time; an order detector fires when part B follows part A; the class node holds that and fires when C arrives. With the same motifs in another order, the “A then B” detector still fires but nothing completes the pattern.



Why learning it is not trivial. When a detector for “A, then B, then C” fails to fire, which of its connections should change? Crediting every candidate spreads the weight so thinly that the node never fires; crediting the tempting shortcut (“A, then B” is shared with another class) traps it; exploring a little, then settling, finds the right order and keeps it (§84).



Where it does not win yet: composition that hinges on fine timing precision (0.99 vs a Transformer's 0.998); spoken digits (0.65 vs 0.70 for a published LSTM); and trading, where no learner beats buy-and-hold on this data.

Sleeping Machines proposes that computation can happen **in time rather than memory**: candidate events race, the first to fire cancels the rest, and what a node computes is set by delays, by how long it holds an input, and by inhibition that arrives in time. This report states what is now known about such networks, why, and what remains open. Derivations and proofs are in experiments/THEORY.md (cited as §n).

Summary

- **What one node computes is exactly characterized, and it says where depth is needed.** A node with a trigger, hold and veto inputs accepts a product set in lag coordinates; one node orders at most three events; two layers compute every conjunction of bounded time differences, three layers every union (§71, proved; exhaustive search agrees). Nodes with state (arm/disarm) are strictly stronger.
- **Event networks learn timing with very few mistakes** if winning is positional (pull the event that should have won, never push a loser later) and order uses held intervals, not aligned delays: $O(\log N + \text{a few})$ mistakes per node.
- **On a timing task a learned event network matches or beats dense models at 10^4 - $10^5\times$ lower cost**, with nothing given: 1.000 at 7.5 synaptic events per episode vs 0.989-0.996 for event-token Transformers at 146k-1.16M multiply-adds and 0.995 for a clocked conv net at 3.07M.
- **Depth is learned natively when credit is right (§83-§86).** Summed potentials with conserved multiplicative credit, credit to one instant with cooled exploration, and synapses grown only when credited (provably the same decisions as dense weights) learn order among three and four parts: 0.999-1.000 on 5/5 seeds at depth 4 with $\approx 2,000$ updates and $\approx 80k$ grown synapses out of $5.5\cdot 10^7$ candidates. Where composition hinges on fine timing precision a Transformer is still more accurate (0.998 vs 0.988-0.995) at $\approx 10^4\times$ the cost and $50\times$ the data.
- **Grokking occurs, by a route change under sleep, and only for relations the substrate can express.** Without sleep the network memorizes; with sleep it generalizes after a delay (0.93-0.99, reliable with cooled timing noise); a data $\times$ sleep phase diagram shows memorization, grokking and collapse; with depth, $(a + b + c) \bmod p$ through two composed rhythm stages reaches 0.99-1.00 (3 seeds, $p = 17$ and 31).
- **Learning cost follows activity, not model size:** 12 $\rightarrow$ 96 input channels leaves learning mistakes flat and makes inference cheaper (§77).
- **Not yet: real asynchronous benchmarks.** SHD below dense baselines; on the market, correctly posed as trading with costs, no learner profits and the native one learns to stay out; the online world model, built as an event network, beats a neural point process by 0.5-0.9 nats per event, also on held-out days.

Where the event paradigm wins, and where it does not

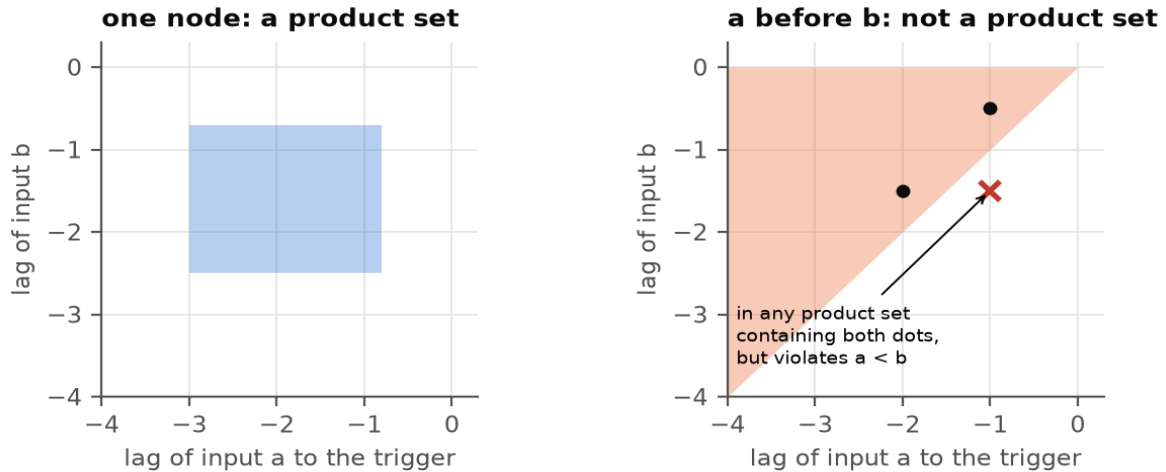
“Supremacy” here means a measured advantage over dense networks given the same data; each row states its caveat.

claim	evidence	caveat
Equal or better accuracy at 10^4-$10^5\times$ lower cost on timing tasks	E35: 1.000 at 7.5 synaptic events, nothing given; conv net 0.995 at 3.07M MACs; event-token Transformer 0.989–0.996 at 146k–1.16M after $10\times$ more training	one task family; much of the gap is the clock (an event-driven conv net $\approx 10\times$)
Groks composed arithmetic where a Transformer does not	E41, $(a + b + c) \bmod 17$, 30% of triples: 0.994–0.999 (3/3) in 200 epochs; Transformer (AdamW, weight decay, 100k steps): 0.29 / 0.63 (seed 0, $d = 32 / 64$), 0.06 / 0.03 (seed 1; chance 0.06)	the two-stage rhythm route is a provided resource (E45: it can choose among routes); more steps might help the Transformer
Better world model of a real market stream at 80–200$\times$ lower cost	E48: online -2.11 vs GRU -2.62 nats/event; held-out frozen $-2.38 / -2.10$ vs $-3.15 / -2.98$; ≈ 19 synaptic ops vs thousands of MACs	classical model class (semi-Markov); offline-trained GRU pending
Learning cost follows activity, not model size	E35: $12 \rightarrow 96$ channels, 0.999–1.000, mistakes flat, inference cheaper ($7.5 \rightarrow 3.2$ – 4.0 synaptic events); \$77, \$81	measured to 96 channels
Structure discovery, implicit Occam razor	E45 pilot: one rhythm for $a + b$ (1.000), the chain for $a + b + c$ (0.999), nothing for random tables	pilot
Deep order learned from few examples	E54: 20 orders of four motifs, 0.999–1.000 (5/5) after 10–15k examples, $\approx 2,000$ updates, ≈ 150 events, $80k$ of $5.5 \cdot 10^7$ candidate synapses grown	Transformer on this task running
<i>Not supremacy</i> : composition hinging on timing precision	E34m chains 0.988–0.995, stable, 40k examples; Transformer 0.998 after 2M (0.42–0.70 after 40k)	Transformer more accurate with $50\times$ the data
<i>Not supremacy</i> : spoken digits (SHD)	E51 class-conditional event world models 0.647 test (0.73 held-in speakers); LSTM ≈ 0.70 ; state of the art ≈ 0.9	unseen test speakers
<i>Not supremacy</i> : trading profit	E42 (21 unseen days): no learner beats buy-and-hold (+932 bp); the priced native one +226 bp, others lose	the data may hold no edge

1. What an event node computes

Primitives. Spike times; delay; first-of (min) and all-of (max); hold (an input opens a window of given duration); veto (an input blocks the node while within its window). A node fires at its trigger if its holds are met and no veto is active.

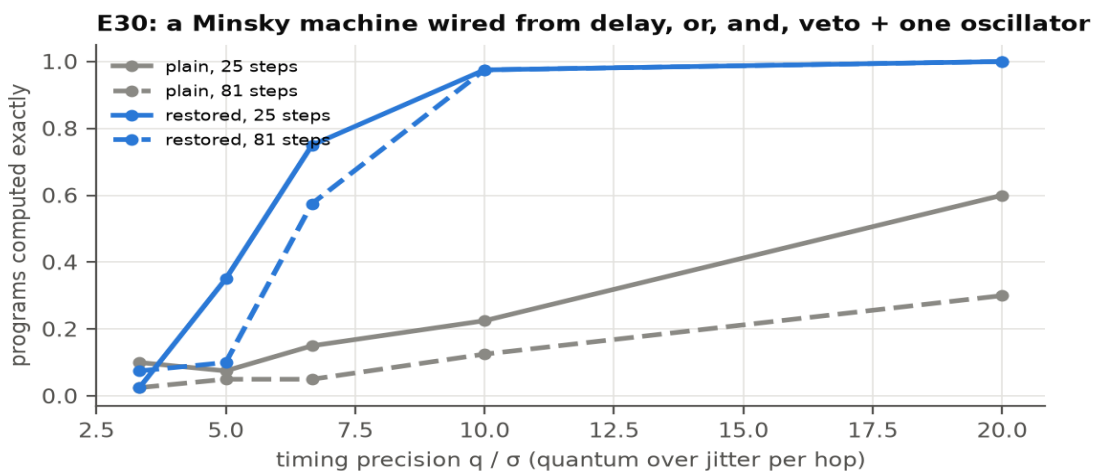
Theorem (§71). In lag coordinates relative to the trigger a stateless node accepts a product set. So one node orders at most three events (trigger the middle one, hold the first, veto the last until it arrives); two layers compute every conjunction of bounded differences with exclusions (every zone of timed-automata verification); three layers every finite union. *Evidence:* an exhaustive search over single nodes finds the proof's own node for 3 events and none for 4 on grids of 1,680 and 11,880 configurations; all thirteen Allen interval relations built at depth 2 are exact on ~19,900 interval pairs (E39).



Stateful nodes are stronger. A node armed by A, disarmed by C and fired by B computes an XNOR of two order relations, which no stateless node computes (exhaustive search, E39b); a toggle node computes parity.

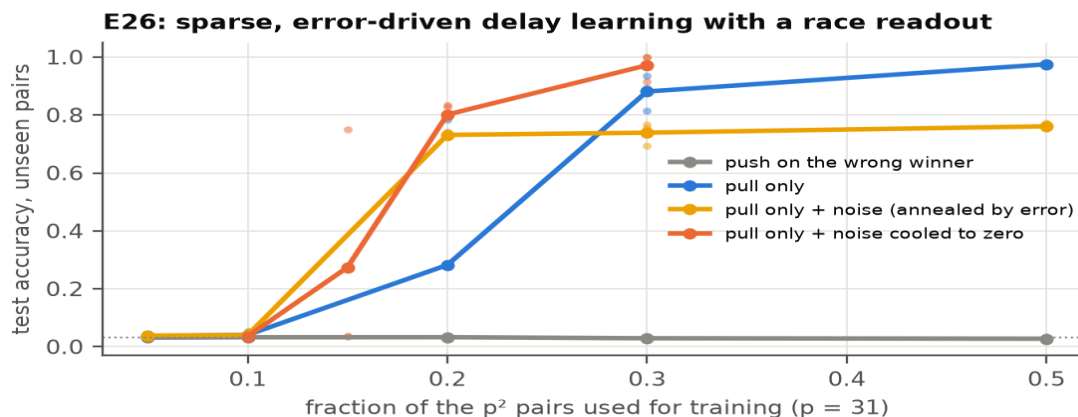
Clockless computation computes relations, not sums: shift symmetry forbids adding two times; one shared rhythm makes cyclic arithmetic computable (§56).

Completeness and reliability (E30). A two-counter machine wired from these primitives plus one oscillator runs exactly (Turing-complete given timing precision). Jitter accumulates like a random walk; one restoring coincidence per cycle makes reliability independent of program length.



2. How event networks learn

Winning is positional (§54). Pull the event that should have won; never push a loser later, which hands the win to the next loser and collapses the detectors. With learned delays on $(a + b) \bmod p$: push at chance in all 15 runs, pull-only 0.97–0.98 (E26). Timing noise cooled to zero makes small-data generalization reliable.



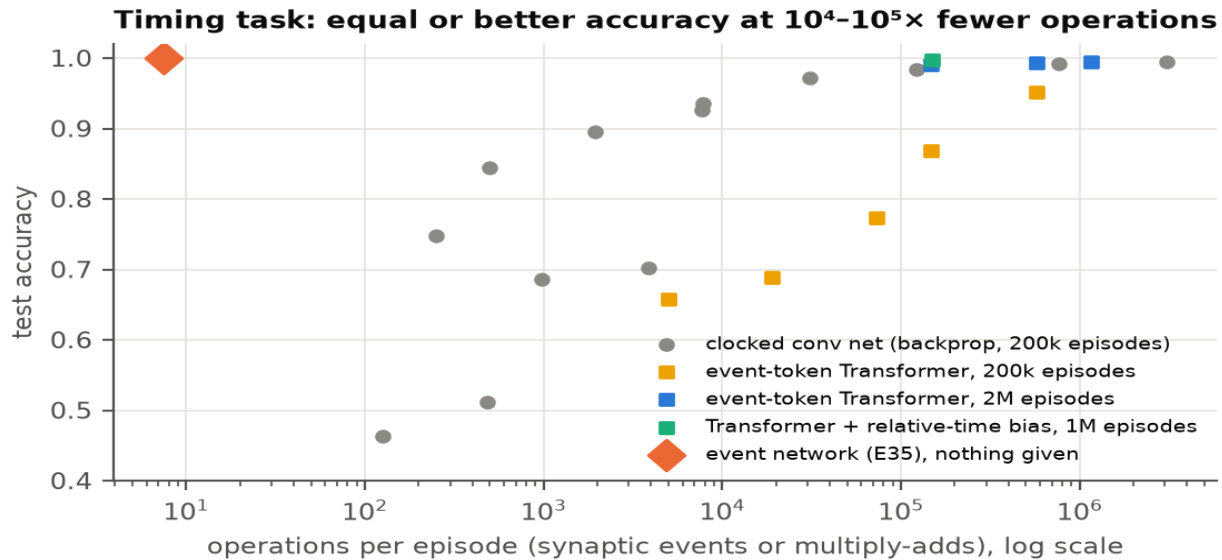
Order needs held intervals, not aligned delays (§61). Delaying A to meet B compresses the interval so a veto cannot see where C fell; A opening a window that B must hit keeps it. With holding, the detector for “B within Δ after A unless C” reaches 1.000 on 5 seeds with ~ 450 updates in 200k episodes, and veto is worth 19 points (alignment: 0.914, veto worth 1.8).

The rules are online learning on the simplex (§68): pulls moving a fixed fraction of a conserved budget are Winnow/Hedge-type updates with $O(k \log N)$ mistake bounds; durations are interval learning, vetoes monotone disjunctions. With nothing given, detectors learn channels, durations and vetoes: 1.000×4 , 0.9995, 443–1,530 updates (E35).

Routing needs counterfactuals (§57). Credit along a spike's causal path cannot say whether another route should have been taken; cancelled near misses supply it. Without it a two-layer network stays at chance (0.17 vs 0.72–0.74, E28).

3. Against dense models and Transformers

model	test accuracy	cost per episode	learning
event network, nothing given (E35)	1.000 ×4, 0.9995	7.5 synaptic events	443-1,530 updates, 200k episodes
event-token Transformer, 2M episodes	0.989-0.996	146k-1.16M MACs	backprop
Transformer + relative-time bias, 1M episodes	0.996, 0.998	≈ 150k MACs	backprop
event-token Transformer, 200k episodes	0.65-0.97	5k-576k MACs	backprop
clocked conv net (E32)	0.995 / 0.984 / 0.895	3.07M / 123k / 1.9k MACs	backprop, 200k

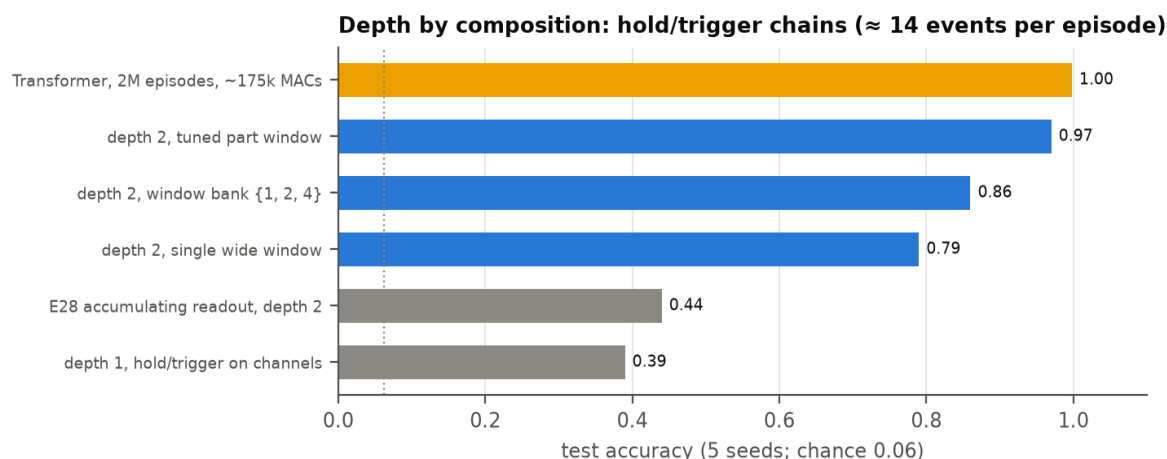


Scaling with the input basis (E35, §77). At fixed spikes per episode, $12 \rightarrow 96$ candidate channels leaves accuracy ≈ 1.0 (96: 1.000, 0.999, 1.000) and learning updates flat while synaptic events per episode fall: learning and inference cost follow activity, not the size of the basis.

Why. Attention cannot see order without position information and must synthesize time comparisons from dot products at $O(n^2 \cdot d)$ per layer; a hold/trigger node computes the comparison as its primitive, at the cost of its input events (§70). The clocked model also pays per time bin, so silence multiplies its cost; a conv net evaluated only where spikes are would cost ≈ 100 multiply-adds, not millions (§63). At published per-operation energies the gap at equal accuracy is $\approx 10^4\times$. *Limits:* one task built around the primitives; the Transformer needed $10\times$ more training for parity; with a learned relative-time attention bias (time differences enter attention directly) it reaches 0.996-0.998 at $\approx 150k$ MACs, still below the event network at $\approx 2 \cdot 10^4\times$ its cost.

4. Depth and composition

Classes that are ordered combinations of shared parts are zones over part events, so the theorem prescribes two layers: parts (hold nodes on channel pairs) and class nodes that hold one part's spike and are triggered by another's, so a single part cannot satisfy them. Learned natively: 0.97 with part windows at the motif scale, 0.86 with a generic bank of window scales, depth 1 0.39 (5 seeds, E34). A readout that accumulates evidence fails at conjunction even though its hidden nodes learn parts (E28). An event-token Transformer trained on 2M episodes reaches 0.998 at $\approx 175\text{k}$ – 690k multiply-adds vs ≈ 14 events. The chains are not training-limited: at 200k episodes they stay at 0.954 (tuned) and 0.87 (bank); some class routes lock onto wrong parts.



What the credit rule must be (§83–§84). A class node should sum its held and its coincident trigger inputs and learn by full-information multiplicative updates under a conserved budget. Proved: conservation resolves the AND's credit ambiguity (a false fire does not say which half was wrong) iff the threshold exceeds half the budget; the mistake bound grows with the log of the candidate basis ($16 \rightarrow 32$ channels: $1.3\times$ the updates). Crediting every candidate deadlocks when units fire in every positive at different instants (window bank: 0.05–0.10); crediting the one instant closest to firing can cycle between invalid prefixes; cooled exploration over instants reaches the valid route, which is absorbing. With a class window covering the task's span, E34's task plateaus at 0.988–0.995 per seed and the generic bank at 0.97–0.99; the gap to 0.998 is timing precision (fixed windows cannot express the minimum intervals).

Order among three parts: depth 3 (E53, E54). Classes that are different orders of the same motif sets (20 classes, decoys = other orders) need ordered intermediates: composite units $u \rightarrow v$ over every ordered pair of parts (57,840 candidates, ≈ 41 events per episode). 5 seeds, 40k episodes:

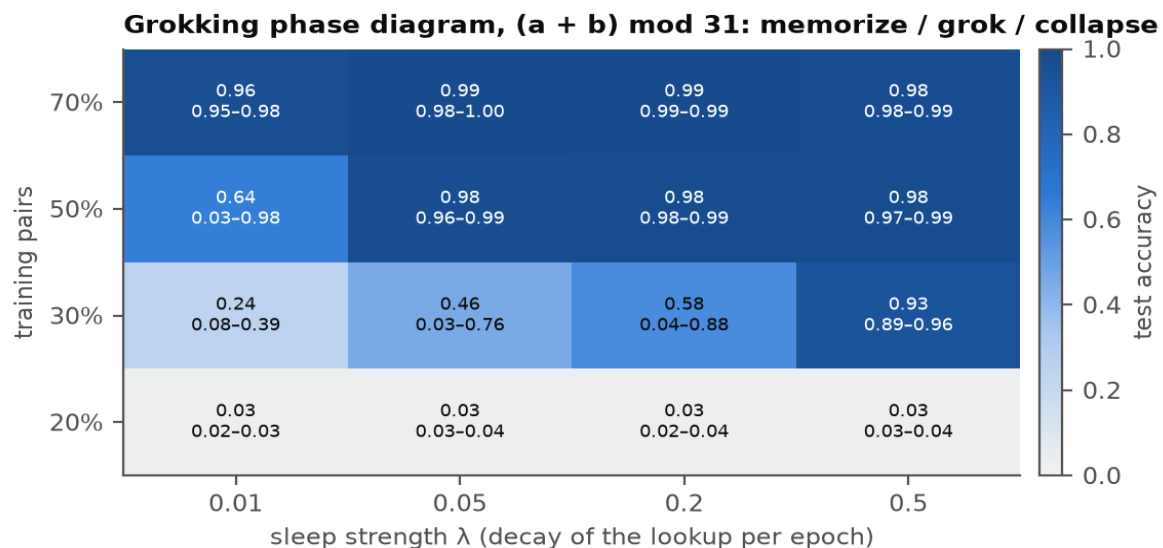
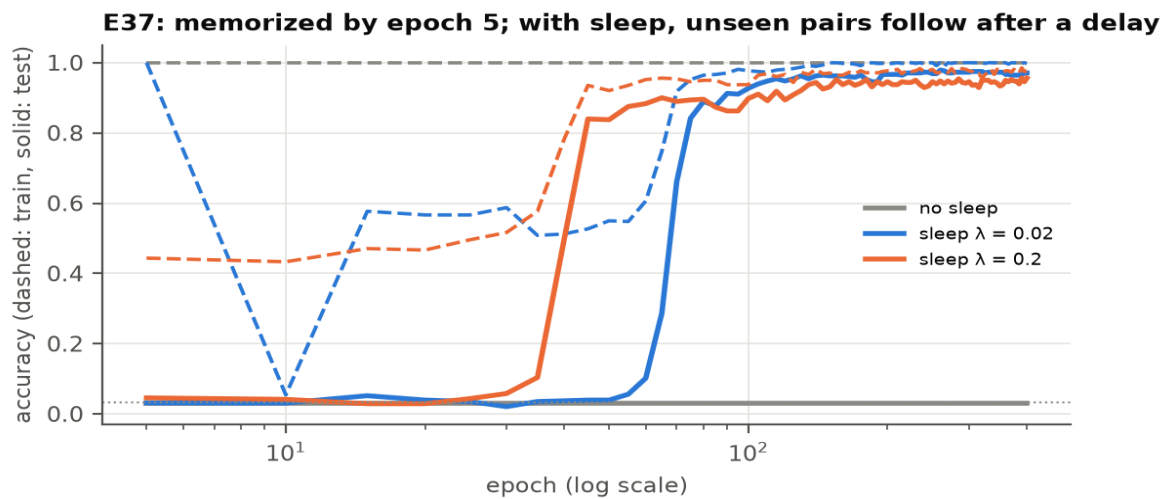
E53	test accuracy	updates
depth 3, + margin earned by reliability (§86b)	0.997–0.999 at every checkpoint, 5/5 seeds	$\approx 1,100$ – $1,400$
depth 3, instant credit, exploration $T = 0.3$	0.998–0.999 on all seeds by 5k episodes; final 0.963–0.999 (dips)	$\approx 1,100$
depth 3, greedy instant credit	0.71–0.85 (cycles between shared prefixes)	4k–11k
depth 3, credit to every candidate	0.33–0.36 (never fires)	$\approx 26\text{k}$
depth 2, same credit	0.32–0.42	$\approx 24\text{k}$

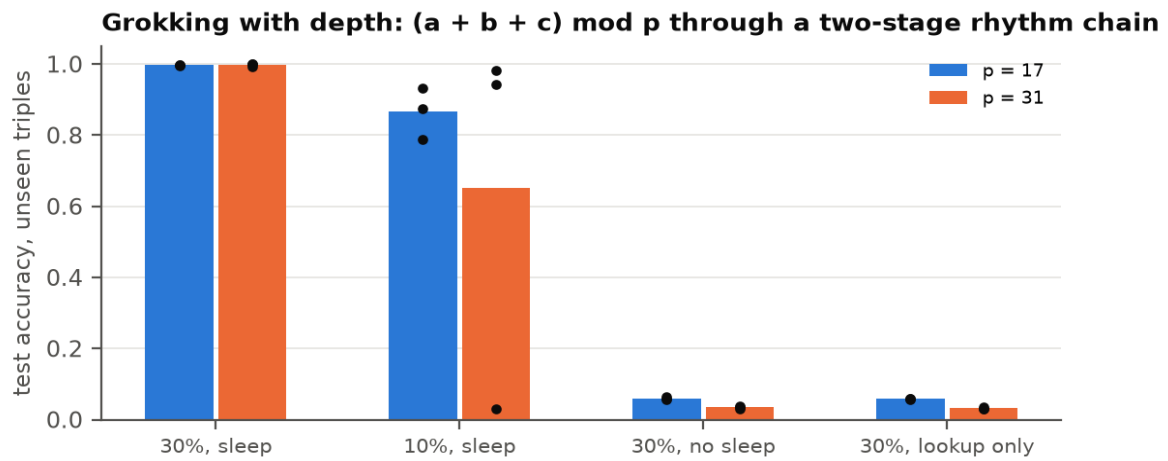
Without a margin, converged classes dip (a node sits just above threshold; one demotion knocks it under). A margin kept by near-miss credit removes the dips but also protects wrong routes (3/5 seeds freeze on a shared prefix); a margin earned by reliability, applied only when the node's recent fires were mostly correct, is stable at 0.997–0.999 (§86b). With synapses grown only when first credited (§85, provably the same decisions as dense weights), depth 3 at 20 channels gives 0.981–1.000 with 9.5k–13.7k of 145k candidate synapses ever grown.

5. Generalization and grokking

What counts (§58): restriction, forced generalization above capacity, and grokking (the relation reached while memorizers are available) are different claims; each reports $p = n/\text{params}$. **Per-class parameters cannot generalize on $(a + b) \bmod p$ (§66):** each operand occurs once per class, so generalization needs shared intermediates; in time, a sum needs a rhythm.

Grokking as a route change (E37). Each class has a pair-node lookup that can memorize everything ($p \approx 0.016$) and a shared route through a rhythm with learned delays; learning is errors-only. Without sleep: train 1.0, test 0.03–0.04. With sleep: test 0.93–0.97 in 2 of 3 seeds without timing noise, and 0.95–0.99 in 5 of 5 seeds with cooled timing noise on the shared route ($\sigma = 2$), after a delay. Sleep without the rhythm: train 0.45, test 0. The rhythm is used only where it fits: $a - b$ and relabelled sums grok (0.95–0.98), $a \cdot b$ in 1 of 3 seeds, while $a^2 + ab + b^2$ and random tables stay at chance on unseen pairs (0.01–0.04) and their training accuracy erodes under sleep. A data $\times$ sleep phase diagram (4×4 , 3 seeds) shows no grokking below 20–30% of pairs, and above it a minimum sleep that falls with data. Larger problems grok more reliably: 2/3 seeds at $p = 31$ and 59, 3/3 at $p = 97$ ($p \approx 0.005$). With depth (E41, 3 seeds): $(a + b + c) \bmod p$ through two composed rhythm stages, 0.99–1.00 on unseen triples from 30% of them at $p = 17$ and 31, and from 10% in 2 of 3 seeds at $p = 31$ ($p \approx 0.003$); chance without sleep; collapse without the chain. The data it needs is far above the Occam bound (§78): at 1–7% of triples it memorizes and stays at chance, also when trained 7–10 $\times$ longer; the threshold at $p = 17$ lies between 7% and 30%.



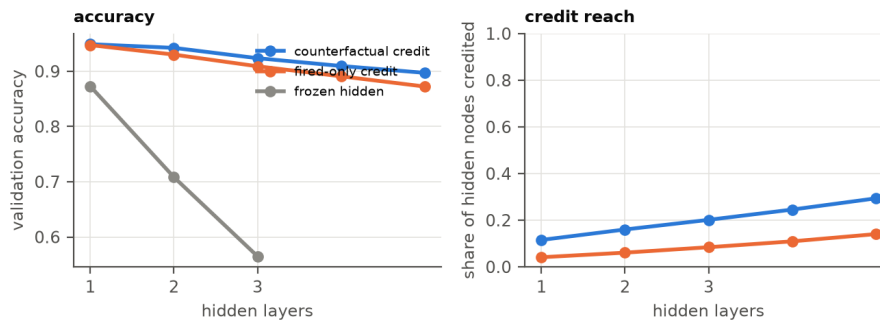


Why (§69, §72). Error-gated learning makes memorization absorbing. Sleep keeps a parameter only if it is used by more than $m^* = \lambda\theta/(e\eta)$ examples: lookup entries serve one and die, the rhythm's delays serve many and survive, and once the rhythm answers a pair its lookup entry is never relearned. This predicts memorization, grokking and collapse regimes (all observed) and a data threshold $n^* \propto p\lambda\theta/(e\eta)$.

6. The weight race

The original architecture (integrate-to-threshold nodes with learned weights, racing in groups) holds its first results: a cancelled node can be taught (E4: 0.79 at $K = 128$ where reward-modulated rules are at chance); races decide as fast as the evidence allows (E2, matching the optimal MSPRT); learning work tracks activity (E5: $\approx 300\times$ fewer updates); local learning reaches ≈ 0.96 on latency-coded MNIST with counterfactual credit that pays more with depth (E6, E14).

E14 · the advantage of counterfactual credit grows with depth (2 seeds)



Its limits: depth still costs accuracy; only its single racing layer beats an equal dense model at inference ($\approx 2.4\times$); it forgets more than SGD (E23); it memorizes instead of grokking (E24); and the rules that fixed the timing networks do not transfer to its weights (SHD 0.04–0.29 vs 0.35).

7. Real data

- **Spiking Heidelberg Digits** (spoken digits as cochlear spike trains, 700 channels, 20 classes, unseen test speakers): class-conditional event world models (E51; state = last spike's band, time since it, time since onset; one counting pass) reach 0.647 test (0.734 on held-in speakers); timing +0.06, onset reference +0.21. The weight race reached 0.35; a published LSTM ≈ 0.70 ; state of the art ≈ 0.9 . SHD is only $\approx 6\times$ sparser than a 10 ms raster, a weak test of the paradigm's cost advantage.
- **Market stream posed as trading with costs (E42, confirmed on 21 unseen days, preregistered)**: at 2 bp, imitating a hindsight teacher over-trades and loses (event learner –15,236 bp, logistic –24,302); the profit-priced event learner nets +226 bp with 8 changes; buy-and-hold +932; at 10 bp all stay out. No learner beats buy-and-hold; pricing the decision is what stops the losses.
- **The world model is an event network, and it beats a neural point process (E44, E48)**. A likelihood decomposition located the GRU's lead in which event comes next; count baselines located the missing information (the time since the last event). A semi-Markov event network (state nodes for the last two types, window nodes from a delay line, count-learned detectors) reproduces that model exactly and scores –2.11 nats per event online (days 1–5) and –2.38 / –2.10 frozen on held-out days 6 / 7, against –2.62 and –3.15 / –2.98 for the GRU, at ≈ 19 synaptic operations per event. The model class is classical; an offline-trained GRU is the remaining check.
- **Earlier native world model (E44), prequential log-likelihood per event (nats; days 1 / 2 / 3)**: Poisson –3.00 / –3.42 / –3.32; Hawkes (Adam) –2.62 / –2.94 / –2.84; native (multiplicative) –2.64 / –2.85 / –2.70; GRU neural point process – / –2.61 / –2.52. Pair-part state neutral; learned inhibition below excitation-only.
- **Online world model (E44)**: a temporal point process of four event types learned from every event; the native model (–2.64/–2.85/–2.70 nats per event) matches Hawkes (–2.62/–2.94/–2.84) and trails a GRU neural point process (–2.61/–2.52) by ≈ 0.2 nats.

8. Open problems and next steps

- Grokking theory tests (running): phase diagram over data $\times$ sleep (§72); relations the rhythm cannot express; sleep as the pressure toward reusable parts (§73); $O(\log N)$ learning as the candidate basis grows (§74).

- Composition accuracy against Transformers; grokking reliability; native learning of sparse parity (\$75, open); a real stream with rare, precisely timed events (\$55).

Every mechanism is an event handler (local state, triggered by events, cost proportional to events); dense procedures are diagnostics only. Reproduce: `python report/figures_time.py` && `python report/make_pdf.py`.